МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

Кафедра вычислительной математики

МЕТОДЫ РЕШИЯ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВНЕНИЙ

Студент: Король Денис Евгеньевич
3 курс 8 группа

Преподаватель: Мандрик Павел Алексеевич

Минск, 2026

Введение

Программы реализованы на языке Java (версия 24, non-LTS) с использованием стандартной библиотеки java.util и java.lang.Math. Компиляция и запуск выполнялись в среде IntelliJ IDEA 2023.2 на платформе OpenJDK 24.0.2. Исходный код программ приведён в приложениях А–Д.

Решение СЛАУ с матрицей Гильберта методом Гаусса «единственного деления»

Матрицей Гильберта называется квадратная матрица A размера $n \times n$, элементы которой задаются формулой

$$a_{ij} = \frac{1}{i+j-1},\ i,j = 1,\dots,n. \qquad (1)$$

Так как в Java индексация массивов начинается с 0, то я изменил формулу

$$a_{ij} = \frac{1}{i+j+1},\ i,j = 0,\dots,n-1. \qquad (2)$$

Метод Гаусса «единственного деления» состоит из двух этапов: прямого хода и обратного хода. На прямом ходе для каждого $i = 0,\dots,n-1$ выполняется деление i-й строки на диагональный элемент a_{ii}, после чего исключаются неизвестные из нижележащих строк. На обратном ходе последовательно вычисляются компоненты вектора решения x от последней к первой.

Для решения СЛАУ с матрицей Гильберта был реализован класс HilbertMatrix (реализация класса в приложении А), формирующий матрицу по формуле (2), и класс GaussianMethod с функциями forwardStroke, выполняющий прямой ход схемы единственного деления, backstroke, выполняющий обратный ход схемы единственного деления, validate, выполняющий проверку входных данных, и solve, вызывающий эти методы для получения итогового ответа. Вектор свободных членов вычислялся в методе matrixVectorMultiply по формуле

$$b = Ax^* \qquad (3)$$

где $x^* = (\dots)^T$ — точное решение.

В моей реализации можно самостоятельно ввести любое точное решение.

Также существуют модификации метода Гаусса с выбором главного элемента по столбцу и по строке, то есть делим не на a_{ii}, а ищем строку где на i-ом месте максимальный элемент и меняем исходную и найденную строки местами, для выбора по сроке ищем максимальный элемент в i-ой строке, и меняем местами столбцы, но при нахождении x они будут перемешаны так же как и строки. Эти методы реализованы в функциях columnSolve и rowSolve класса GaussianMethod.

Все функции класса GaussianMethod приведены в приложении Б.

Нахождение норм для векторов ошибки

Вектор ошибки находится по формуле

$$err_i = \|x - x^*\|_i,\ i = 0, \dots, n-1 \qquad (4)$$

где x — найденное численное решение, x^* — точное решение.

Кубическую норму находил по формуле

$$\|err\| = max|err_i|,\ i = 0, \dots, n-1 \qquad (5)$$

Октаэдрическую норму находил по формуле

$$\|err\| = \sum_{i=0}^{n-1}|err_i| \qquad (6)$$

Сферическую норму находил по формуле

$$\|err\| = \sqrt{\sum_{i=0}^{n-1} err_i^2} \qquad (7)$$

Вектор ошибки был найден по формуле (4) с помощью ранее описанных функций и функции VectorDifference.
Нормы были найдены с помощью функций cubicNorm, octahedralNorm, sphericalNorm.
Эти функции приведены в приложении В.

Нормы векторов ошибки для найденных решений СЛАУ для $n = 3, 7, 11, 15, 19$

Таблица 1— нормы вектора ошибки для $n = 3$

Метод	Кубическая норма	Октаэдрическая норма	Сферическая норма
Единственного деления	7,54951657e-15	1,57651669e-14	1,04266434e-14
С выбором по столбцу	1,25455202e-14	2,68673972e-14	1,76347316e-14
С выбором по строке	1,02140518e-14	2,19824159e-14	1,43986981e-14

Таблица 2— нормы вектора ошибки для $n = 7$

Метод	Кубическая норма	Октаэдрическая норма	Сферическая норма
Единственного деления	2,53114991e-08	7,21045835e-08	3,70768809e-08
С выбором по столбцу	2,24577184e-08	6,39667204e-08	3,28821151e-08
С выбором по строке	1,96606594e-08	5,59895563e-08	2,87740970e-08

Таблица 3— нормы вектора ошибки для $n = 11$

Метод	Кубическая норма	Октаэдрическая норма	Сферическая норма
Единственного деления	1,43868194e-02	4,99807671e-02	2,27690727e-02
С выбором по столбцу	1,31960695e-02	4,58522180e-02	2,08837812e-02
С выбором по строке	1,28148767e-02	4,45164115e-02	2,02818392e-02

Таблица 4— нормы вектора ошибки для $n = 15$

Метод	Кубическая норма	Октаэдрическая норма	Сферическая норма
Единственного деления	7,40261180e+00	3,23643689e+01	1,26626849e+01
С выбором по столбцу	1,08587072e+01	4,48399105e+01	1,80097345e+01
С выбором по строке	7,24036742e+00	2,56061814e+01	1,13636841e+01

Таблица 5— нормы вектора ошибки для $n = 19$

Метод	Кубическая норма	Октаэдрическая норма	Сферическая норма
Единственного деления	4,61489487e+01	2,45369386e+02	8,40511500e+01
С выбором по столбцу	2,71427961e+01	1,33496094e+02	4,66024354e+01
С выбором по строке	1,29898148e+01	7,66538054e+01	2,55659632e+01

Анализируя каждую таблицу, можно заметить, что выбор главного элемента по строке является наиболее устойчивой модификацией метода Гаусса для нахождения этих норм, однако принципиальное ограничение точности связано с плохой обусловленностью самой матрицы Гильберта.

Квадратная матрица с диагональным доминированием

Матрицей с диагональным доминированием называется квадратная матрица *A* размера $n \times n$, если для каждой строки выполняется условие

$$|a_{ii}| > \sum_{i \neq j} |a_{ij}|, \ i, j = 0, \dots, n-1. \qquad (8)$$

Программа просто создает матрицу нужного размера, заполняет ее случайными значениями, а потом заменяет диагональные элементы суммой модулей всех элементов строки, сохраняя изначальный знак.
Реализация приведена в приложении Г.

Число обусловленности заданной матрицы

Число обусловленности матрицы A находится по формуле

$$\nu(A) = \|A\| * \|A^{-1}\| \qquad (9)$$

где $\|\cdot\|$ — некоторая матричная норма

Кубическую норму находил по формуле

$$\|A\| = max \sum_{j=0}^{n-1} |a_{ij}|, \ i = 0, \dots, n-1 \qquad (10)$$

Октаэдрическую норму находил по формуле

$$\|A\| = max \sum_{i=0}^{n-1} \left|a_{ij}\right|,\ j = 0, \dots, n-1 \qquad (11)$$

Сферическую норму находил по формуле

$$\|A\| = \sqrt{\sum_{i=0}^{n-1} \sum_{j=0}^{n-1} a_{ij}^2} \qquad (12)$$

Для этого задания реализована функция нахождения обратной матрицы, функции нахождения норм матриц по указанным выше формулам и функции вычисления числа обусловленности, все они показаны в приложении Д.

ПРИЛОЖЕНИЕ А

Матрица Гильберта

```
public class HilbertMatrix implements Matrix{
    private final int size;
    private final double[][] matrix;

    public HilbertMatrix(int n) {
        size = n;
        if (size <= 0) {
            throw new IllegalArgumentException("Size must be positive, got: " +
size);
        }
        matrix = new double[size][size];

        for (int i = 0; i < size; i++) {
            for (int j = 0; j < size; j++) {
                matrix[i][j] = 1.0 / (i + j + 1);
            }
        }
    }

    @Override
    public double[][] getMatrix() {
        double[][] copy = new double[size][size];
        for (int i = 0; i < size; i++) {
            copy[i] = matrix[i].clone();
        }
        return copy;
    }

    @Override
    public int getSize() {
        return size;
    }

}
```

ПРИЛОЖЕНИЕ Б

Метод Гаусса

```
public class GaussianMethod {

    private static void forwardStroke(double[][] matrix, double[] rhs, int i,
double eps) {
        int size = matrix.length;
        double aii = matrix[i][i];
        if (Math.abs(aii) < eps) {
            throw new ArithmeticException("Matrix is singular or nearly singular
at column " + i);
        }
        for (int j = i; j < size; j++) {
            matrix[i][j] /= aii;
            if (Double.isInfinite(matrix[i][j]) || Double.isNaN(matrix[i][j])) {
                throw new ArithmeticException("Division by zero");
            }
        }
        rhs[i] /= aii;
        if (Double.isInfinite(rhs[i]) || Double.isNaN(rhs[i])) {
            throw new ArithmeticException("Division by zero");
        }
        for (int k = i + 1; k < size; k++) {
            double aki = matrix[k][i];
            for (int t = i; t < size; t++) {
                matrix[k][t] -= (matrix[i][t] * aki);
            }
            rhs[k] -= (rhs[i] * aki);
        }
    }

    private static void backStroke(double[][] matrix, double[] rhs) {
        int size = matrix.length;
        for (int i = size - 1; i >= 0; i--) {
            for (int k = i - 1; k >= 0; k--) {
                rhs[k] -= (rhs[i] * matrix[k][i]);
                matrix[k][i] = 0;
            }
        }
    }

    public static double[] solve(double[][] matrix, double[] rhs, double eps) {
        validate(matrix, rhs);

        int size = matrix.length;
        double[][] a = new double[size][size];
        for (int i = 0; i < size; i++) {
            a[i] = matrix[i].clone();
        }
        double[] x = rhs.clone();

        for (int i = 0; i < size; i++) {
            forwardStroke(a, x, i, eps);
        }
        backStroke(a, x);
        return x;
    }

    public static double[] columnSolve(double[][] matrix, double[] rhs, double
```

```
eps) {
        validate(matrix, rhs);

        int size = matrix.length;
        double[][] a = new double[size][size];
        for (int i = 0; i < size; i++) {
            a[i] = matrix[i].clone();
        }
        double[] x = rhs.clone();
        int pivotRow;
        double maxAbs;
        for (int i = 0; i < size; i++) {
            pivotRow = i;
            maxAbs = Math.abs(a[pivotRow][pivotRow]);

            for(int c = i + 1; c < size; c++) {
                if (Math.abs(a[c][i]) > maxAbs) {
                    pivotRow = c;
                    maxAbs = Math.abs(a[c][i]);
                }
            }

            if (pivotRow != i) {
                double[] bufferStr = a[i].clone();
                a[i] = a[pivotRow];
                a[pivotRow] = bufferStr;

                double bufferX = x[i];
                x[i] = x[pivotRow];
                x[pivotRow] = bufferX;
            }

            forwardStroke(a, x, i, eps);
        }

        backStroke(a, x);
        return x;
    }

    public static double[] rowSolve(double[][] matrix, double[] rhs, double eps)
{
        validate(matrix, rhs);

        int size = matrix.length;
        double[][] a = new double[size][size];
        for (int i = 0; i < size; i++) {
            a[i] = matrix[i].clone();
        }
        double[] x = rhs.clone();

        int pivotColumn;
        double maxAbs;
        int[] perVec = new int[size];
        for (int i = 0; i < size; i++) {
            perVec[i] = i;
        }

        for (int i = 0; i < size; i++) {
            pivotColumn = i;
            maxAbs = Math.abs(a[pivotColumn][pivotColumn]);

            for(int c = i + 1; c < size; c++) {
                if (Math.abs(a[i][c]) > maxAbs) {
```

```
                    pivotColumn = c;
                    maxAbs = Math.abs(a[i][c]);
                }
            }

            if (pivotColumn != i) {
                double tempElem;
                int tempInd;
                for (int j = 0; j < size; j++) {
                    tempElem = a[j][i];
                    a[j][i] = a[j][pivotColumn];
                    a[j][pivotColumn] = tempElem;
                }
                tempInd = perVec[i];
                perVec[i] = perVec[pivotColumn];
                perVec[pivotColumn] = tempInd;
            }

            forwardStroke(a, x, i, eps);
        }
        backStroke(a, x);
        double tempX;
        int tempPer;
        for (int i = 0; i < size; i++) {
            if (perVec[i] != i) {
                tempX = x[i];
                x[i] = x[perVec[i]];
                x[perVec[i]] = tempX;

                tempPer = perVec[perVec[i]];
                perVec[perVec[i]] = perVec[i];
                perVec[i] = tempPer;
                i--;
            }
        }

        return x;
    }

    private static void validate(double[][] matrix, double[] rhs) {
        if (matrix == null || rhs == null) {
            throw new IllegalArgumentException("RSH or/and matrix must not be
null");
        }

        int n = matrix.length;
        if (n == 0) {
            throw new IllegalArgumentException("Matrix must not be empty");
        }

        if (rhs.length != n) {
            throw new IllegalArgumentException("RSH and matrix sizes must be
equal");
        }

        for (double[] doubles : matrix) {
            if (doubles.length != n) {
                throw new IllegalArgumentException("Matrix must be square");
            }
        }
    }
}
```

ПРИЛОЖЕНИЕ В

Векторные нормы

```
public static double[] vectorsDifference(double[] mas1, double[] mas2) {
    if (mas1 == null || mas2 == null) {
        throw new IllegalArgumentException("X or X* must not be null");
    }

    int n = mas1.length;
    if (n == 0) {
        throw new IllegalArgumentException("Matrix must not be empty");
    }

    if (mas2.length != n) {
        throw new IllegalArgumentException("RSH and matrix sizes must be
equal");
    }

    double[] err = new double[n];

    for (int i = 0; i < n; i++) {
        err[i] = mas1[i] - mas2[i];
    }
    return err;
}

public static double cubicNorm(double[] mas) {
    double maxAbs = Math.abs(mas[0]);
    for (double ma : mas) {
        if (maxAbs < Math.abs(ma)) {
            maxAbs = Math.abs(ma);
        }
    }
    return maxAbs;
}

public static double octahedralNorm(double[] mas) {
    double sum = 0;
    for (double ma : mas) {
        sum += Math.abs(ma);
    }
    return sum;
}

public static double sphericalNorm(double[] mas) {
    double sum = 0;
    for (double ma : mas) {
        sum += ma * ma;
    }
    return Math.sqrt(sum);
}
```

ПРИЛОЖЕНИЕ Г

Квадратная матрица с диагональным доминированием

```
public class DiagonallyDominationMatrix implements Matrix {
    private final int size;
    private final double[][] matrix;
    private final static int maxAbs = 10000;

    public DiagonallyDominationMatrix(int n) {
        size = n;
        if (size <= 0) {
            throw new IllegalArgumentException("Size must be positive, got: " +
size);
        }
        matrix = new double[size][size];

        for (int i = 0; i < size; i++) {
            double sumAbs = 0;
            for (int j = 0; j < size; j++) {
                matrix[i][j] = ((Math.random() * (maxAbs * 2 + 1)) - maxAbs) /
100.0;
                sumAbs += Math.abs(matrix[i][j]);
            }
            matrix[i][i] = (matrix[i][i] >= 0) ? sumAbs : sumAbs * (-1);
        }
    }

    @Override
    public double[][] getMatrix() {
        double[][] copy = new double[size][size];
        for (int i = 0; i < size; i++) {
            copy[i] = matrix[i].clone();
        }
        return copy;
    }

    @Override
    public int getSize() {
        return size;
    }
}
```

Число обусловленности матрицы

```
private static void forwardStroke(double[][] matrix, double[][] invMatrix, int
i, double eps) {
    int size = matrix.length;
    double aii = matrix[i][i];
    if (Math.abs(aii) < eps) {
        throw new ArithmeticException("Matrix is singular or nearly singular at
column " + i);
    }
    for (int j = 0; j < size; j++) {
        matrix[i][j] /= aii;
        invMatrix[i][j] /= aii;
        if (Double.isInfinite(matrix[i][j]) || Double.isNaN(matrix[i][j])) {
            throw new ArithmeticException("Division by zero");
        }
    }

    for (int k = i + 1; k < size; k++) {
        double aki = matrix[k][i];
        for (int t = 0; t < size; t++) {
            matrix[k][t] -= (matrix[i][t] * aki);
            invMatrix[k][t] -= (invMatrix[i][t] * aki);
        }
    }
}

private static void backStroke(double[][] matrix, double[][] invMatrix) {
    int size = matrix.length;
    for (int i = size - 1; i >= 0; i--) {
        for (int k = i - 1; k >= 0; k--) {
            for (int j = 0; j < size; j++) {
                invMatrix[k][j] -= (invMatrix[i][j] * matrix[k][i]);
            }
            matrix[k][i] = 0;
        }
    }
}

public static double[][] inverseMatrix(double[][] matrix, double eps) {
    if (matrix == null) {
        throw new IllegalArgumentException("Matrix must not be null");
    }

    int n = matrix.length;
    if (n == 0) {
        throw new IllegalArgumentException("Matrix must not be empty");
    }

    for (double[] doubles : matrix) {
        if (doubles.length != n) {
            throw new IllegalArgumentException("Matrix must be square");
        }
    }

    int size = matrix.length;
    double[][] a = new double[size][size];
    double[][] invA = new double[size][size];
    for (int i = 0; i < size; i++) {
        a[i] = matrix[i].clone();
```

```
            invA[i][i] = 1;
        }

        int pivotRow;
        double maxAbs;

        for (int i = 0; i < size; i++) {
            pivotRow = i;
            maxAbs = Math.abs(a[pivotRow][pivotRow]);

            for(int c = i + 1; c < size; c++) {
                if (Math.abs(a[c][i]) > maxAbs) {
                    pivotRow = c;
                    maxAbs = Math.abs(a[c][i]);
                }
            }

            if (pivotRow != i) {
                double[] bufferStr = a[i].clone();
                a[i] = a[pivotRow];
                a[pivotRow] = bufferStr;

                bufferStr = invA[i].clone();
                invA[i] = invA[pivotRow];
                invA[pivotRow] = bufferStr;
            }

            forwardStroke(a, invA, i, eps);
        }

        backStroke(a, invA);
        return invA;
    }

    public static double cubicNorm(double[][] matrix) {
        double maxSumAbs = 0;
        for (double[] row : matrix) {
            double sumAbs = 0;
            for ( double elem : row) {
                sumAbs += Math.abs(elem);
            }
            if (sumAbs > maxSumAbs) {
                maxSumAbs = sumAbs;
            }
        }
        return maxSumAbs;
    }

    public static double octahedralNorm(double[][] matrix) {
        double maxSumAbs = 0;
        for (int i = 0; i < matrix[0].length; i++) {
            double sumAbs = 0;
            for (int j = 0; j < matrix.length; j++) {
                sumAbs += Math.abs(matrix[j][i]);
            }
            if (sumAbs > maxSumAbs) {
                maxSumAbs = sumAbs;
            }
        }
        return maxSumAbs;
    }

    public static double sphericalNorm(double[][] matrix) {
```

```
    double sumAbs = 0;
    for (int i = 0; i < matrix.length; i++) {
        for (int j = 0; j < matrix[i].length; j++) {
            sumAbs += matrix[i][j] * matrix[i][j];
        }
    }
    return Math.sqrt(sumAbs);
}

public static double cubicCond(double[][] matrix, double eps) {
    return MatrixUtils.cubicNorm(matrix)
            * MatrixUtils.cubicNorm(MatrixUtils.inverseMatrix(matrix, eps));
}

public static double octahedralCond(double[][] matrix, double eps) {
    return MatrixUtils.octahedralNorm(matrix)
            * MatrixUtils.octahedralNorm(MatrixUtils.inverseMatrix(matrix,
eps));
}

public static double sphericalCond(double[][] matrix, double eps) {
    return MatrixUtils.sphericalNorm(matrix)
            * MatrixUtils.sphericalNorm(MatrixUtils.inverseMatrix(matrix, eps));
}
```